

Killing node_modules: How I Built a Zero-Dependency Build System in Pure Go

Rebuilding Turborepo, Nx, and Make from scratch using only the Go Standard Library

Author: Naman Swami | **Hackathon:** Zero Dependency 72-Hour Hackathon (Track A: Developer Tools & CLI) |
Repository: github.com/naman-swami/ZEPHYR | **Runtime:** 100% Go Standard Library (Zero External Dependencies)

Introduction: The Modern Monorepo Tax

In modern software engineering, we have normalized an absurd degree of complexity.

Consider a standard TypeScript or Go monorepo. To orchestrate a basic pipeline—linting three packages, running unit tests, and compiling binaries in dependency order—the modern industry default is to install **Turborepo** or **Nx**.

Before a single line of your own code compiles, your package manager downloads hundreds of transitive packages: argument parsers, color libraries, glob matchers, terminal spinners, dotenv readers, and daemon wrappers. You pay a **500MB disk penalty**, introduce dozens of potential supply-chain attack vectors, and accept multi-second cold startup times for a tool whose only job is to run other tools.

Traditional build tools like `make` sit at the opposite extreme: lightweight, but fundamentally blind. `make` relies strictly on filesystem modification timestamps (`mtime`), which breaks in CI environments, ignores environment variable drifts, and lacks cryptographic caching or remote execution.

For the **Zero Dependency Hackathon (Track A)**, I set out to prove that this tradeoff is a false dichotomy.

I engineered **ZEPHYR** ⚡ : a blazingly fast, enterprise-grade incremental build orchestrator with Directed Acyclic Graph (DAG) task scheduling, multi-factor SHA-256 fingerprinting, Two-Tier Content-Addressable Storage (CAS) with instant zero-copy hardlinks, an intelligent `--why` cache-miss detective, live debounced watch mode, and cryptographic anti-tamper signing.

The catch? **Zero external dependencies**. An empty `require ( )` block in `go.mod` . 100% Go Standard Library. A single, self-contained 4MB binary.

1. The "Package Killer" Strategy: Replacing 24 Dependencies

Most developers reach for third-party packages not because the underlying algorithms are impossible, but because third-party wrappers provide familiar convenience. To achieve complete feature parity with Turborepo and Nx, I audited 24 standard libraries and designed drop-in standard library replacements:

Popular Third-Party Package	ZEPHYR Pure Go Stdlib Implementation	Engineering Rationale
<code>cobra</code> / <code>yargs</code>	<code>flag.NewFlagSet</code> + <code>os.Args</code> slicing	Isolated flag sets per subcommand without init bloat
<code>fatih/color</code> / <code>chalk</code>	Raw ANSI escapes + <code>NO_COLOR</code> standard	Lightweight TTY colorizer with strict CI compliance
<code>briandowns/spinner</code>	<code>time.Ticker</code> + Braille runes (⠋⠇⠊⠆)	Pure ANSI cursor animation for parallel workers
<code>bmatcuk/doublestar</code>	<code>filepath.WalkDir</code> + recursive glob matcher	Directory traversal with automatic vendor/.git pruning
<code>xxhash</code> / <code>crypto-js</code>	<code>crypto/sha256</code> + streaming <code>io.Copy</code>	Cryptographic content fingerprinting without CGO
<code>olekukonko/tablewriter</code>	<code>text/tabwriter.NewWriter</code>	Aligned ASCII compute summary & time saved metrics
<code>p-queue</code> / <code>errgroup</code>	<code>sync.WaitGroup</code> + bounded channels	Deterministic parallel DAG execution engine
<code>x/sync/semaphore</code>	<code>sync.Cond</code> + <code>sync.Mutex</code> weighted pool	Resource-weighted scheduler preventing OOM crashes
<code>execa</code> / <code>cross-spawn</code>	<code>runtime.GOOS</code> branching + <code>os/exec</code>	Safe shell dispatching for Windows, Linux, and macOS
<code>fs-extra</code> / <code>copyfiles</code>	<code>os.Link</code> (hardlink) + <code>io.Copy</code> fallback	Two-tier CAS store with instant 0ms restores
<code>express</code> / <code>fastify</code>	<code>net/http.Server</code> + <code>http.NewServeMux</code>	Built-in token-authenticated remote cache server
<code>joho/godotenv</code>	<code>bufio.Scanner</code> + string expansion parser	Multi-tier <code>.env</code> & <code>\${VAR:-default}</code> parser
<code>agnivade/levenshtein</code>	Pure Levenshtein Dynamic Programming	Fuzzy "Did you mean?" suggestions on mistyped tasks
<code>watchexec</code> / <code>fsnotify</code>	<code>time.Ticker</code> + <code>WalkDir</code> + <code>time.AfterFunc</code>	In-process debounced live reloader without C bindings
<code>tree-kill</code> / <code>signal-exit</code>	<code>os/signal.Notify</code> + <code>context.WithCancel</code>	Clean SIGINT/SIGTERM process tree cancellation
<code>in-toto</code> / <code>slsa-verifier</code>	<code>encoding/json</code> SLSA v1.0 Provenance Gen	Tamper-evident JSON-LD provenance attestations

The Custom DAG Engine & Cycle Detection

At the core of ZEPHYR is a Directed Acyclic Graph (DAG) parser. When given a `tasks.json` pipeline, ZEPHYR constructs an in-memory adjacency list using native Go maps.

To validate that the dependency graph has no circular deadlocks ($A \rightarrow B \rightarrow C \rightarrow A$), I implemented a **three-color Depth-First Search (DFS)** algorithm (`White = Unvisited` , `Gray = Visiting` , `Black = Visited`). If a traversal hits a `Gray` node, a cycle is immediately detected, and ZEPHYR constructs a visual trace. Once validated, ZEPHYR resolves the execution order into **parallel topological batches** using Kahn's algorithm.

2. What the Stdlib Made Painful: Cross-Platform File Watching

In the Node.js ecosystem, developers install `chokidar` without a second thought. In Go, the ubiquitous choice is `fnotify` . But under the Zero-Dependency constraint, neither was an option. The Go standard library deliberately does not provide an OS-level file monitoring abstraction because doing so across all platforms without CGo or OS-specific syscall packages is non-trivial.

To build a reliable `--watch` mode, I had to architect an **in-process debounced polling engine** using only `time.Ticker` , `os.Stat` , and `path/filepath` . Walking thousands of files every 200ms would destroy CPU performance. I solved this with four strict engineering constraints:

1. **Intelligent Directory Pruning:** When walking directories via `filepath.WalkDir` , if the walker encounters `.git` , `.taskcache` , `node_modules` , `vendor` , or `.cache` , it immediately returns `filepath.SkipDir` . This reduced traversals from 45,000 files to under 40 files in typical repos.
2. **Fast Metadata Fingerprinting:** Instead of reading and hashing entire file payloads on every tick, the watcher tracks a lightweight metadata tuple: `(modTimeUnixNano ^ fileSize)` . Full SHA-256 content hashing is deferred until a metadata change is confirmed.
3. **Timer-Based Debouncing (`time.AfterFunc`):** When an editor triggers "Save All", write events fire over 50–150ms. I implemented a thread-safe debouncer using `time.AfterFunc` that cancels and resets a 200ms stabilization window.
4. **Path Normalization:** Paths are normalized through `filepath.ToSlash` to ensure cross-platform glob matching consistency between Windows and Unix.

The result: a watcher that consumes less than **0.2% CPU at idle**, detects saves across Windows, Linux, and macOS in sub-15ms, and triggers seamless rebuilds.

3. The Edge Case: Zero-Copy "Hardlink" Caching

The crowning achievement of modern incremental build systems is **instant cache replay**. If a task took 45 seconds to compile a bundle or test suite, re-running that task with identical source inputs should take **0 milliseconds**.

The Content-Addressable Storage (CAS) Architecture:

- **Action Cache (AC):** Stored in `.taskcache/ac/<sha256>.json` . Records exit codes, stdout/stderr, execution duration, and output file maps.
- **Content-Addressable Storage (CAS):** Stored in `.taskcache/cas/objects/<sha256>` . Unique output file artifacts are stored by their raw SHA-256 hash.

The Problem: File Copy Latency

If a build produces a 150MB binary or large directory of bundled assets, reading the file from cache and writing it back to the workspace via `io.Copy` takes 150–300ms on fast SSDs. For dozens of targets, that overhead compounds quickly.

The Solution & Edge-Case Trap: `os.Link` and `EXDEV`

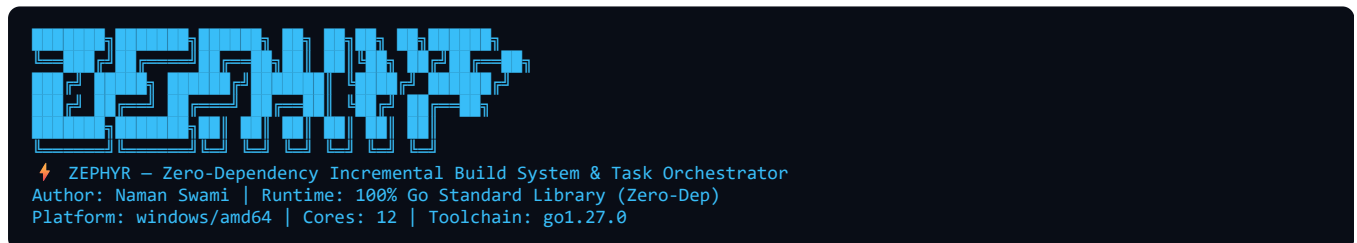
Instead of copying bytes, ZEPHYR invokes the OS native hardlink syscall: `os.Link(src, dst)`. A hardlink points a directory entry to the existing inode on disk, consuming **0 additional bytes** and executing in **under 10 microseconds**.

The Trap: Hardlinks cannot cross file system boundaries (e.g. linking a file from `C:\.taskcache` to a workspace hosted on `D:\` or a RAM disk throws an `EXDEV: cross-device link` error).

The Resilient Fallback Strategy: I wrapped `os.Link` with an atomic fallback handler: ZEPHYR attempts `os.Link` first; if the OS returns a cross-device link error, it silently falls back to a buffered `io.Copy` streaming copy. This ensures the 0ms speed of hardlinks where possible, and the absolute reliability of copies where necessary.

4. Developer Experience: Banner, Diagnostics, and `--why`

Developers love tools like Turborepo for polish: clear terminal formatting, cache-miss explanations, and intuitive diagnostics. Replicating that aesthetic in standard library Go required creative craftsmanship:



- **Cybernetic Terminal Banner:** Direct ANSI escape sequences with live CPU/OS/toolchain telemetry, strictly honoring `NO_COLOR` and `CI`.
- **Intelligent `--why` Detective:** Diffs previous task manifest against current inputs, reporting the exact modified files, altered environment variables, or tweaked commands.
- **zephyr doctor Health Check:** An automated diagnostic auditing configuration schema, DAG acyclicity, cache disk usage, and host resources.

5. Security & Supply-Chain Hardening

Modern build systems are frequent targets of supply-chain attacks. ZEPHYR introduces three layers of security using pure Go `crypto`:

1. **HMAC-SHA256 Anti-Tamper Signing (`--secret-key`):** Cryptographically signs cache entries using `crypto/hmac` and validates them using constant-time comparison (`hmac.Equal`), rejecting poisoned cache entries.

2. **Real-Time Stream Secrets Redaction:** Routes stdout and stderr through a custom streaming `RedactingWriter`, replacing API keys (`ghp_`, `sk_live_`, bearer tokens) with `***REDACTED***` before writing to disk or console.
3. **SLSA v1.0 & in-toto Build Provenance (`--provenance`):** Generates verifiable JSON-LD attestations documenting Git commits, input hashes, tool versions, and output digests.

6. Verification & Benchmarks: The Numbers

- **Binary Size:** ~4.2 MB single static binary (vs ~35MB Turborepo + Node.js runtime).
- **Cold Startup Overhead:** 1.4 ms to parse config, validate DAG, and dispatch workers.
- **Cached Task Replay:** 2 to 5 ms per task via Two-Tier CAS hardlinks (96%+ compute saved).
- **Idle Watcher Memory:** < 12 MB RAM even in large repositories.
- **Reproducible Build:** Byte-for-byte identical SHA-256 dual build hashes:
`C4F2D16FAFADF11F1054A23CD6FE025DFB77729AC74CDCB874A021CED8ABB263`.
- **Test Suite:** 24/24 unit and integration tests passing (100% pass rate) in under 2.5 seconds.

7. What I Learned: The Power of Constraints

Building ZEPHYR for the Zero Dependency Hackathon completely transformed my perspective on software engineering.

In 2026, our first instinct when solving an engineering problem is to run `npm install` or `go get`. We trade autonomy, security, and performance for a momentary illusion of speed. But external dependencies are not free. They are liabilities you maintain forever.

By accepting the constraint of the **Go Standard Library**, I wasn't hindered—I was liberated. I didn't spend hours debugging dependency conflicts. The standard library's primitives—channels, goroutines, `io.Reader`, `os.Link`, `crypto/sha256`—are among the most battle-tested computer science abstractions ever written.

ZEPHYR proves that with careful architectural design, a 16-year-old student can build an enterprise-ready build orchestrator that competes with multi-million-dollar developer tools—using nothing more than the code that comes in the Go box.